

Vom ersten Python-Skript zum eigenen Handelsmodell

Eine 12-Monats-Roadmap für Jannes: Python, Mathematik, Statistik und Machine Learning — aufgebaut um ein einziges Projekt, dein ICT-Modell.

12

Monate

6

Phasen

10,5 h

pro Woche

555 h

gesamt

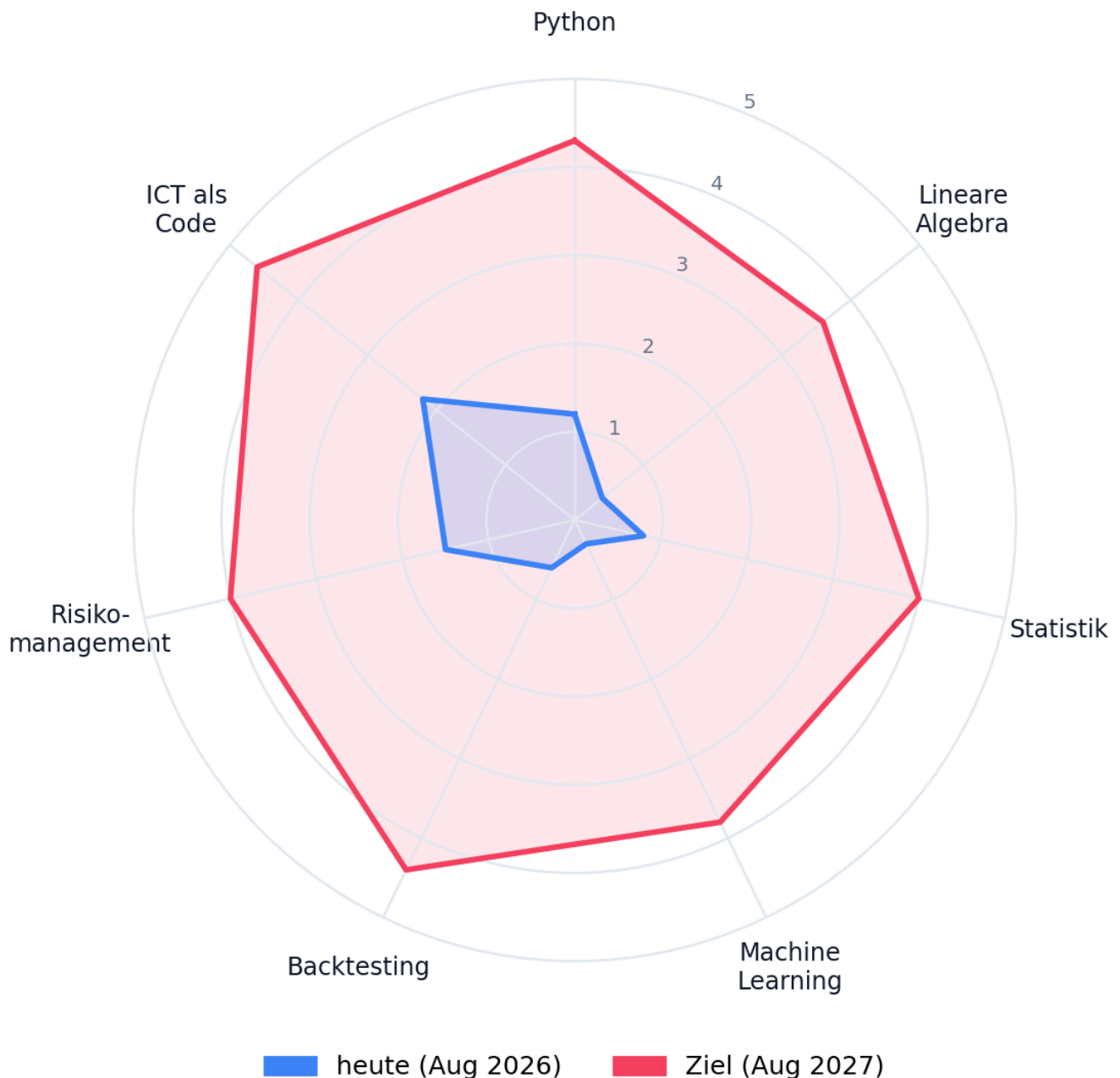
Dein Ziel, in einem Satz:

Selbstständig und zielstrebig arbeiten können — ohne bei jedem zweiten Schritt nachschlagen zu müssen. Dieser Plan ist genau darauf gebaut: nicht auf Zertifikate, sondern auf Können, das im Kopf bleibt.

Wo du stehst und wo du hinwillst

Ehrliche Standortbestimmung, damit der Plan realistisch bleibt.

Du hast rund eine Stunde Python hinter dir und stehst bei `int()`. Das ist kein Nachteil — es ist ein sauberer Startpunkt. Der eigentliche Unterschied zwischen jemandem, der Charts anschaut, und einem Quant liegt nicht im Code, sondern in einer Frage: **Kannst du beweisen, dass dein Vorteil echt ist und nicht Zufall?** Alles in diesem Plan arbeitet auf die Fähigkeit hin, diese Frage zu beantworten.



Selbsteinschätzung heute gegen Zielprofil in 12 Monaten. Die Skala ist bewusst auf „arbeitsfähig ohne Hilfe“ geeicht, nicht auf „Universitätsniveau“.

Was „sicher arbeiten“ konkret bedeutet

Damit das Ziel messbar wird, hier die Definition, an der du dich am Ende misst. In 12 Monaten kannst du:

- ein leeres Python-Projekt aufsetzen, versionieren und testen — ohne Anleitung;

- Kursdaten laden, bereinigen und ein ICT-Konzept in eine Funktion übersetzen, die man testen kann;
- lesen, was in einer Formel mit Matrizen, Erwartungswerten oder Gradienten steht, statt sie zu überspringen;
- ein ML-Modell trainieren *und* begründen, warum sein Ergebnis nicht durch Data Leakage entstanden ist;
- einen Backtest so bauen, dass du ihm selbst misstraust — und ihn trotzdem verteidigen kannst.

Wichtige Erwartung vorweg. „Nicht mehr nachschlagen müssen" heißt nicht, dass du irgendwann alles auswendig weißt. Auch erfahrene Entwickler schlagen täglich Syntax nach. Der Unterschied: Sie wissen *genau, wonach* sie suchen und erkennen die Antwort in fünf Sekunden. Dieses Gefühl ist das echte Ziel — und es entsteht durch Wiederholung und eigene Projekte, nicht durch mehr Kurse.

Was in solchen Plänen meistens fehlt

Du hast gefragt, ob Themen fehlen. Ja — und zwar genau diese sechs.

Fast jeder, der sich Quant-Wissen selbst beibringt, plant Python, Mathematik und Machine Learning ein. Diese sechs Bereiche werden fast immer vergessen — und sie sind der Grund, warum Modelle im Backtest glänzen und live sterben.

Die sechs blinden Flecken

Fast jeder Autodidakt lernt Python, Mathe und ML — und übersieht diese sechs. Sie sind in der Roadmap eingeplant.

Software-Handwerk Git, virtuelle Umgebungen, pytest, Projektstruktur, Logging. <i>Ohne das schreibst du Code, den du in 3 Monaten nicht mehr verstehst.</i>	Zeitreihen-Statistik Stationarität, ADF-Test, Autokorrelation, Volatilitätsclustering (GARCH). <i>Kursdaten sind KEINE unabhängigen Stichproben. Standard-Statistik lügt hier.</i>	Datenqualität & Bias Look-ahead, Survivorship, Repainting, Zeitzonen, Tick vs. Bar, Splits. <i>Der häufigste Grund für Backtests, die live nicht funktionieren.</i>
Marktmikrostruktur Spread, Slippage, Ordertypen, Order Flow, Liquidität, Kosten. <i>ICT IST Mikrostruktur — nur ohne die Fachbegriffe dahinter.</i>	Backtest-Overfitting Multiple Testing, Deflated Sharpe, Purged & Embargoed CV. <i>Der Unterschied zwischen einem Modell und einer schön gerechneten Illusion.</i>	Risiko & Ausführung Positionsgröße, Kelly, Drawdown, Broker-API, Monitoring, Ausfälle. <i>Entscheidet über dein Konto — nicht die Trefferquote.</i>

Warum gerade diese sechs für dich zählen

Zeitreihen sind ein Sonderfall

Praktisch jede Statistik-Einführung setzt voraus, dass Datenpunkte unabhängig voneinander sind. Kurse sind das nicht: Der Preis von heute hängt am Preis von gestern, Volatilität kommt in Clustern. Wer das ignoriert, hält Zufall für Signal. Deshalb bekommt Zeitreihenanalyse hier einen eigenen Block statt einer Fußnote.

ICT ist Mikrostruktur ohne Fachbegriffe

Liquidity Sweeps, Order Blocks, Fair Value Gaps beschreiben Orderfluss und Liquidität. In der Literatur heißen dieselben Phänomene Order Imbalance, Stop-Hunting und Price Impact. Wer diese Sprache dazulernt, kann seine Konzepte messbar machen — und findet Forschung, die sonst verschlossen bleibt.

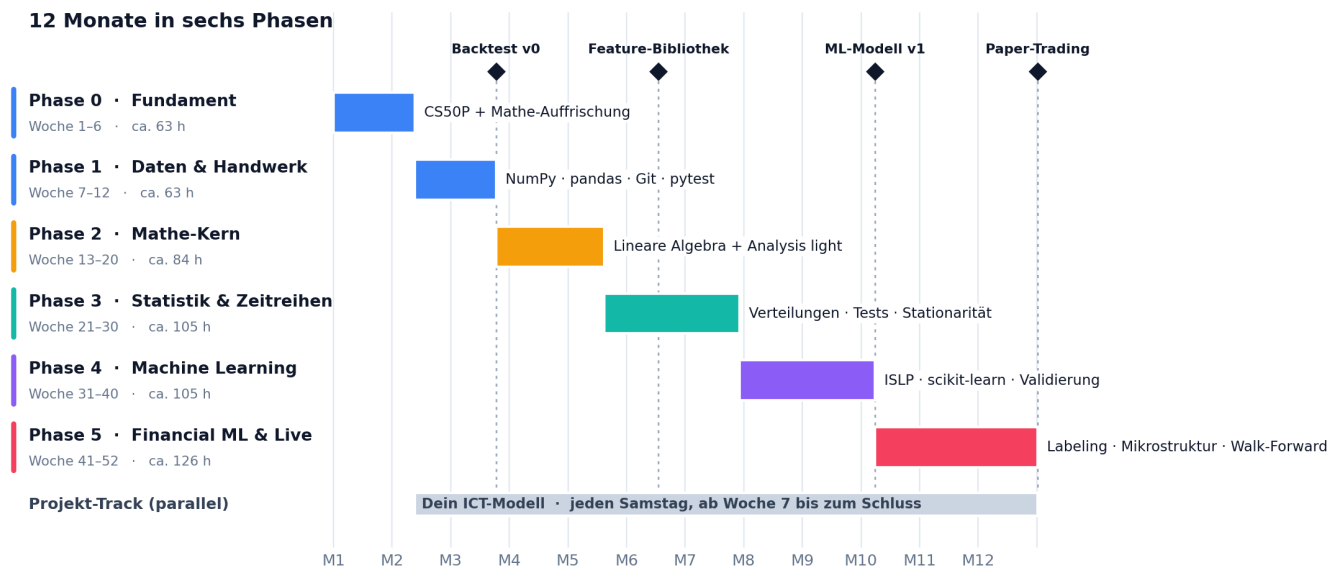
Der teuerste blinde Fleck: Backtest-Overfitting. Wenn du 200 Parameterkombinationen durchprobierst, findest du garantiert eine mit toller Equity-Kurve — auch bei reinem Zufallsrauschen. Ohne Purged Cross-Validation und eine Korrektur für die Anzahl der Versuche (Deflated Sharpe Ratio) ist ein guter Backtest schlicht keine Information. Phase 5 dreht sich zur Hälfte um genau dieses Problem.

Bewusst *nicht* im Plan

Vollständigkeit ist der Feind eines Plans, der neben einem Job funktioniert. Nicht nötig für dein Ziel und deshalb bewusst gestrichen: stochastische Analysis (Itô-Kalkül, Black-Scholes), Optionspreistheorie, Deep Learning, C++ und Low-Latency-Programmierung, Maßtheorie. Für den Weg Richtung Fonds oder Bank kommt das obendrauf — für dein eigenes Modell brauchst du es nicht.

Der Fahrplan über 12 Monate

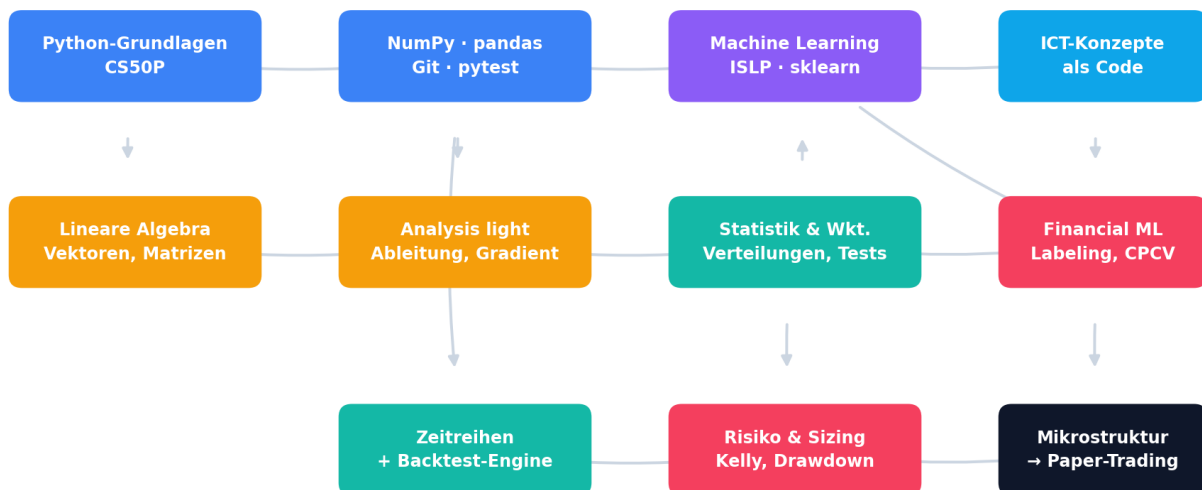
Sechs Phasen, vier Meilensteine, ein durchgehendes Projekt.



Das Prinzip: Du lernst nie länger als zwei Monate am Stück Theorie, ohne sie anzuwenden. Der Projekt-Track läuft ab Woche 7 parallel — jeden Samstag baust du an deinem eigenen Modell weiter, mit dem Wissen, das du gerade frisch hast. Dadurch festigt sich Gelerntes automatisch, und du siehst permanent, wozu du das alles machst.

Abhängigkeiten: was worauf aufbaut

Kein Kasten funktioniert ohne die Kästen links davon. Reihenfolge ist kein Vorschlag.



Die Reihenfolge ist kein Geschmack. Statistik ohne Analysis bleibt Rezeptwissen, Machine Learning ohne lineare Algebra bleibt Copy-Paste, und Financial ML ohne Statistik ist gefährlich.

Warum Mathematik erst ab Woche 13 in die Tiefe geht. Mathe ohne Werkzeug zum Ausprobieren bleibt abstrakt. Sobald du NumPy beherrschst, kannst du jede Matrixoperation sofort selbst nachrechnen und sehen. Das halbiert die Zeit, die lineare Algebra erfahrungsgemäß braucht — und die Auffrischung der Schulmathematik in Phase 0 läuft ohnehin schon vorher.

Nacheinander oder parallel? Die wichtigste Planungsfrage

Die naheliegende Idee ist: erst Python vollständig beherrschen, dann Mathematik, dann Statistik, dann ML. Das klingt sauber — und ist der häufigste Weg, wie solche Pläne scheitern. Aus drei Gründen:

- **„Vollständig sicher“ hat keine Ziellinie.** Es gibt keinen Tag, an dem du mit Python fertig bist. Wer auf dieses Gefühl wartet, wartet für immer und landet in der Tutorial-Falle. Deshalb arbeitet dieser Plan mit *Schwellen* statt mit Perfektion: Du gehst weiter, wenn der Selbsttest der Phase bestanden ist — auch wenn du dich noch unsicher fühlst.
- **Reines Blocklernen wird schneller vergessen.** Wenn du acht Wochen nur Mathe machst und danach acht Wochen nichts mehr, ist das meiste weg. Zwei Themen abwechselnd innerhalb einer Woche fühlen sich anstrengender an und bleiben nachweislich besser hängen.
- **Motivation braucht Anwendung.** Sechs Monate Theorie ohne einen einzigen eigenen Chart ist der schnellste Weg zum Abbruch — gerade neben einem Job.

Die Antwort ist deshalb **beides, aber gestaffelt**: Innerhalb jeder Woche laufen zwei bis drei Themen parallel (Code, Mathe, Projekt). Über die Monate hinweg gibt es aber eine klare Reihenfolge, welches Thema gerade das *Hauptthema* ist. Was **nicht** parallel geht, sind die harten Abhängigkeiten aus der Grafik oben: Ohne Python kein NumPy, ohne lineare Algebra kein sinnvolles ML, ohne Statistik keine belastbare Validierung. Diese Reihenfolge ist bindend.

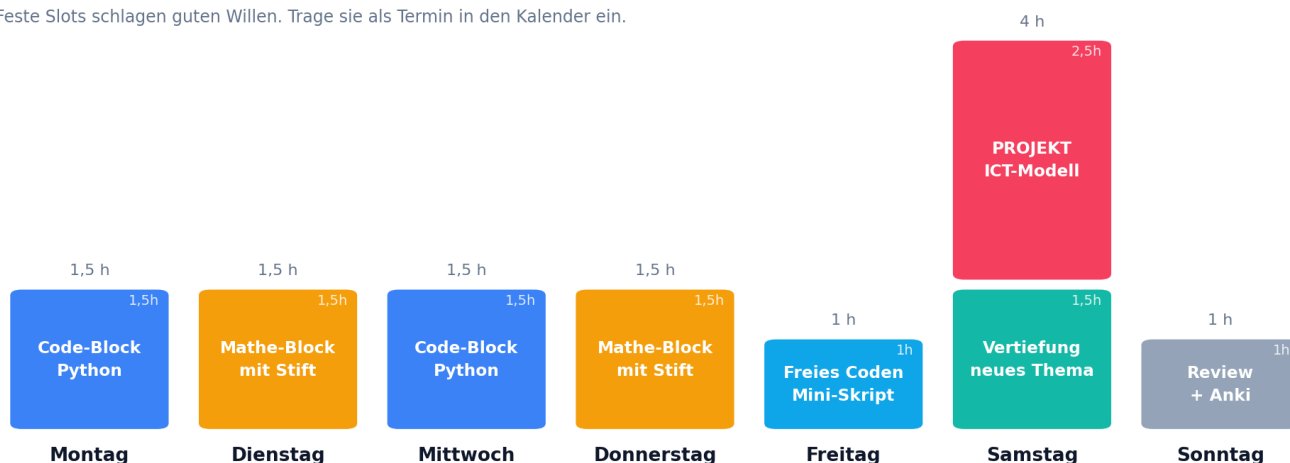
Die eine Ausnahme: Phase 0 ist tatsächlich fast reines Python plus Mathe-Auffrischung, ohne Projekt. Sechs Wochen lang brauchst du erst einmal genug Grundlage, um überhaupt etwas bauen zu können. Ab Woche 7 läuft der Projekt-Track dann durchgehend mit — und hört bis zum Schluss nicht mehr auf.

Deine Woche

Der Plan steht und fällt mit dieser einen Seite.

Wochenrhythmus · ca. 10,5 Stunden

Feste Slots schlagen guten Willen. Trage sie als Termin in den Kalender ein.



Warum Code und Mathe getrennt sind

Beides im selben Block zu mischen kostet dich jedes Mal 15 Minuten Umschaltzeit. Montag/ Mittwoch gehört dem Rechner, Dienstag/ Donnerstag Stift und Papier — Mathematik lernst du nicht durch Lesen, sondern durch eigenes Rechnen. Bei 1,5 Stunden schaffst du 6–8 Aufgaben, das reicht.

Warum der Sonntag heilig ist

Eine Stunde, in der nichts Neues dazukommt: Anki-Karten der Woche durchgehen, dann der Blank-Page-Test (siehe Kapitel 7). Das ist die Stunde, die entscheidet, ob du in sechs Monaten noch weißt, was du im Monat 2 gelernt hast. Sie fällt als letztes aus, nie als erstes.

Die vier Regeln, die den Rhythmus schützen

- **Termin statt Vorsatz.** Trag die Blöcke als wiederkehrenden Kalendereintrag ein. Ein Slot, der nicht im Kalender steht, findet nicht statt.
- **Immer im selben Zustand aufhören.** Beende jede Session mit einer Notiz „hier weiter“. Das spart beim nächsten Mal 10 Minuten Wiedereinstieg.
- **Ausgefallen ist ausgefallen.** Wenn eine Woche wegfällt, wird nichts nachgeholt. Du steigst dort wieder ein, wo du aufgehört hast, und der Plan verschiebt sich um eine Woche. Aufhol-Marathons sind der häufigste Grund, warum Lernpläne sterben.
- **Minimum-Tag statt Nulltag.** An schlechten Tagen: 15 Minuten Anki. Das hält die Kette am Leben — und die Kette ist wichtiger als jeder einzelne Tag.



- Python & Daten · 110 h
- Mathematik · 90 h
- Statistik · 80 h
- Machine Learning · 95 h
- Projekt (ICT-Modell) · 130 h
- Review & Wiederholung · 50 h

Beachte: Der größte Einzelposten ist das Projekt, nicht die Theorie. Das ist Absicht.

Die sechs Phasen im Detail

Jede Phase mit Themen, Ressourcen und einem Selbsttest zum Abhaken.

Phase 0 · Fundament

Woche 1–6 · ca. 63 Stunden · Python-Grundlagen und Schulmathematik reaktivieren

PYTHON (MO/MI/FR + SA)

- CS50P Woche 0–4: Funktionen, Bedingungen, Schleifen, Exceptions, Bibliotheken
- Alle Problem Sets selbst lösen — keine Lösungen auf GitHub ansehen
- Werkzeuge einrichten: VS Code, Python lokal, Anki

MATHEMATIK (DI/DO)

- Bruch- und Potenzrechnung, Gleichungen nach jeder Variable umstellen
- Logarithmus und die Zahl e — warum Renditen logarithmiert werden
- Funktionen, Steigung, Prozent- und Zinsrechnung
- Summenzeichen Σ lesen können (Grundlage jeder Statistikformel)

SELBSTTEST AM ENDE VON WOCHE 6

Du schreibst eine Funktion mit Parametern, Rückgabewert und try/except aus dem Kopf. Du löst $r = (P_1/P_0)^{1/n} - 1$ nach n auf, ohne zu googeln. Du erklärst in einem Satz, warum log-Renditen addierbar sind und einfache Renditen nicht.

Phase 1 · Daten und Handwerk

Woche 7–12 · ca. 63 Stunden · Vom Skript zum Projekt

PYTHON VERTIEFEN

- CS50P Woche 5–9: Unit Tests, File I/O, Regex, OOP
- CS50P-Abschlussprojekt — nimm etwas aus dem Trading-Umfeld
- NumPy: Arrays, Vektorisierung, Broadcasting
- pandas: DataFrame, Index, groupby, resample, merge, Zeitzonen
- matplotlib: Kursverläufe und Equity-Kurven plotten

BLINDER FLECK 1: HANDWERK

- Git und GitHub: commit, branch, diff, revert
- Virtuelle Umgebungen, requirements.txt, Projektstruktur
- pytest: erste eigene Tests für eigene Funktionen
- Jupyter zum Forschen, .py-Dateien für alles, was bleiben soll

SELBSTTEST + MEILENSTEIN „BACKTEST V0“ (WOCHE 13)

Ein Repository auf GitHub mit sauberen Commits, das historische OHLCV-Daten lädt, eine banal einfache Regel anwendet (z. B. „kaufe beim Bruch des Vortageshochs“) und eine Equity-Kurve zeichnet. Bewusst simpel — es geht um die Pipeline, nicht um Profitabilität. Du beherrschst groupby und resample ohne Nachschlagen.

Phase 2 · Mathe-Kern

Woche 13–20 · ca. 84 Stunden · Lineare Algebra und so viel Analysis wie nötig

LINEARE ALGEBRA

- Vektoren, Skalarprodukt, Norm — und was „Korrelation ist ein Winkel“ bedeutet
- Matrizen als lineare Abbildungen, Matrixmultiplikation als Verkettung
- Determinante, Inverse, Rang, lineare Gleichungssysteme
- Eigenwerte und Eigenvektoren
- Kovarianzmatrix und PCA — der direkte Bezug zu Portfolios

ANALYSIS, NUR DAS NOTIGE

- Ableitung als Änderungsrate, Ketten- und Produktregel
- Partielle Ableitung und Gradient
- Minimum/Maximum finden — die Grundlage jedes Trainings
- Gradient Descent von Hand rechnen, dann in NumPy bauen

Arbeitsweise: Erst 3Blue1Brown für die Anschauung, dann Khan Academy zum Rechnen, dann dasselbe in NumPy nachbauen. Diese Dreierschleife ist der schnellste Weg von „verstanden“ zu „kann ich“.

SELBSTTEST AM ENDE VON WOCHE 20

Du erklärst ohne Hilfsmittel, warum Matrixmultiplikation nicht kommutativ ist. Du berechnest eine 3×3 -Determinante ohne Formelsammlung. Du implementierst Gradient Descent für eine einfache Funktion in unter 20 Zeilen NumPy. Du berechnest die Kovarianzmatrix mehrerer Assets und liest ab, was die Eigenvektoren bedeuten.

Phase 3 · Statistik und Zeitreihen

Woche 21–30 · ca. 105 Stunden · Die wichtigste Phase des ganzen Jahres

WAHRSCHEINLICHKEIT UND STATISTIK

- Bedingte Wahrscheinlichkeit, Satz von Bayes
- Erwartungswert, Varianz, Standardabweichung, Schiefe, Kurtosis
- Normal-, t- und Binomialverteilung — und warum Kursrenditen *keine* Normalverteilung haben (Fat Tails)
- Stichprobe, Konfidenzintervall, Hypothesentest, p-Wert und seine Grenzen
- Bootstrapping und Monte-Carlo-Simulation
- Lineare Regression: OLS, R^2 , Residuen, Multikollinearität

BLINDER FLECK 2: ZEITREIHEN

- Stationarität und der ADF-Test
- Autokorrelation, ACF und PACF
- Random Walk — die Nullhypothese, gegen die du antrittst
- Log-Renditen, Volatilitätsclustering, GARCH (Intuition genügt)
- Kennzahlen: Sharpe, Sortino, Max Drawdown, Profit Factor, Expectancy

SELBSTTEST + MEILENSTEIN „FEATURE-BIBLIOTHEK“ (WOCHE 25)

Ein getestetes Python-Modul, das deine ICT-Konzepte erkennt: Swing Points, Break of Structure, Fair Value Gaps, Order Blocks, Liquidity Sweeps — jeweils mit pytest-Tests gegen Beispiel-Charts. Und du kannst erklären, warum ein Sharpe Ratio von 2,0 nichts wert ist, wenn du 500 Varianten getestet hast.

Phase 4 · Machine Learning

Woche 31–40 · ca. 105 Stunden · Modelle bauen und ihnen misstrauen

GRUNDLAGEN

- Supervised Learning, Trainings-/Test-Trennung
- Bias-Varianz-Abwägung, Over- und Underfitting
- Cross-Validation — und warum die Standardvariante bei Zeitreihen falsch ist
- Regularisierung: Ridge und Lasso
- Logistische Regression als Basismodell

MODELLE UND BEWERTUNG

- Entscheidungsbäume, Random Forest, Gradient Boosting (LightGBM/XGBoost)
- Feature Importance und SHAP-Werte
- Precision, Recall, F1, ROC/AUC, Konfusionsmatrix
- Unausgeglichene Klassen — dein Standardfall im Trading
- scikit-learn Pipelines und wie sie Data Leakage verhindern

Die eine Sache, die hier zählt: Data Leakage. Wenn beim Skalieren, beim Feature-Bauen oder beim Splitten auch nur eine Information aus der Zukunft in die Trainingsdaten sickert, sieht dein Modell brillant aus und ist wertlos. Bei Kursdaten passiert das fast automatisch, wenn man nicht aufpasst.

SELBSTTEST + MEILENSTEIN „ML-MODELL V1“ (WOCHE 41)

Ein Klassifikationsmodell, das deine ICT-Setups nach Erfolgswahrscheinlichkeit sortiert, verglichen gegen ein dummes Basismodell. Und: Du findest Data Leakage in fremdem Notebook-Code, ohne dass jemand dich darauf hinweist.

Phase 5 · Financial ML und der Weg live

Woche 41–52 · ca. 126 Stunden · Wo sich entscheidet, ob dein Modell echt ist

FINANCIAL MACHINE LEARNING

- Bar-Typen: Time, Tick, Volume und Dollar Bars
- Triple-Barrier-Labeling statt naivem „steigt/fällt“
- Meta-Labeling: erst Richtung, dann Positionsgröße
- Sample Weights bei überlappenden Labels
- Purged K-Fold mit Embargo, Combinatorial Purged CV
- Deflated Sharpe Ratio und Multiple-Testing-Korrektur

BLINDE FLECKEN 3-6

- Mikrostruktur: Spread, Slippage, Ordertypen, Session-Liquidität
- Realistische Kostenmodellierung im Backtest
- Positionsgröße: Volatilitätsskalierung, Kelly und Fractional Kelly
- Drawdown-Grenzen und Korrelation zwischen mehreren Strategien
- Broker-API, Paper-Trading, Logging, Monitoring, Ausfallszenarien
- Datenqualität: Look-ahead, Survivorship, Repainting, Zeitzonen

SELBSTTEST + MEILENSTEIN „PAPER-TRADING“ (WOCHE 53)

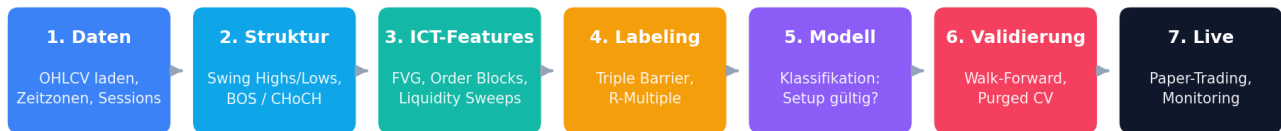
Dein Modell läuft live auf einem Demokonto, mit Logging und wöchentlichem Report. Der eigentliche Test ist aber ein Gespräch: Du kannst begründen, warum dieses Modell live funktionieren sollte — ohne dabei auf die Backtest-Kurve zu zeigen.

Der Projekt-Track: dein ICT-Modell

Das rote Band, das alle Phasen zusammenhält.

Dein Projekt-Track: vom ICT-Chart zum getesteten Modell

Jeder Schritt braucht das Wissen aus der jeweiligen Lernphase. Du baust ihn, sobald du dort ankommst.



Faustregel: Schritt 6 entscheidet, ob dein Modell echt ist. 90 % der Retail-Backtests scheitern genau hier.

Jeden Samstag arbeitest du 2,5 Stunden an genau dieser Kette. Du baust immer nur den Schritt, für den du das Wissen schon hast — und verbesserst rückwirkend die früheren Schritte, sobald du mehr kannst. Nach zwölf Monaten hast du kein Kurszertifikat, sondern ein Repository, das deine Arbeit belegt.

Wie du ICT-Konzepte übersetzbar machst

Der entscheidende Schritt ist Schritt 3. Ein ICT-Konzept ist erst dann brauchbar, wenn es sich als Funktion schreiben lässt, die auf jedem Chart dasselbe Ergebnis liefert. Beispiel Fair Value Gap: „ein Ungleichgewicht“ ist keine Definition — „die Kerze $i-1$ hat ein Hoch, das unter dem Tief von Kerze $i+1$ liegt, und die Lücke ist größer als $x \cdot \text{ATR}$ “ schon. Diese Übersetzung ist echte Quant-Arbeit, und dabei merkst du schnell, welche Konzepte präzise sind und welche nur nachträglich am Chart eingezeichnet wurden.

Die drei Fragen pro Konzept

1. Kann ich es als Code schreiben, ohne den Chart zu sehen?
2. Ist es nur mit Daten der Vergangenheit berechenbar?
3. Tritt es oft genug auf, um statistisch etwas auszusagen?

Ein Konzept, das eine dieser Fragen nicht besteht, gehört nicht ins Modell.

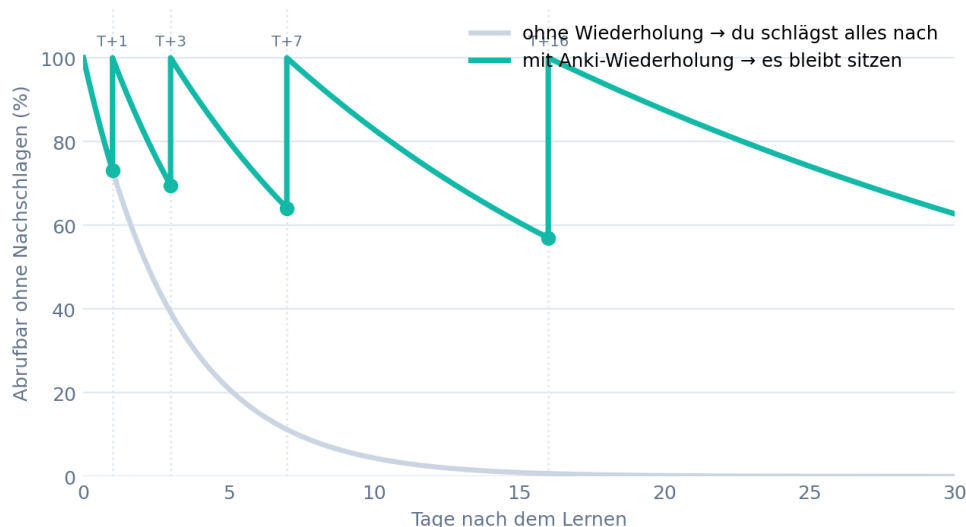
Realistische Erwartung

Wenn du deine Regeln sauber testest, wird ein Teil davon nicht halten. Das ist kein Scheitern — das ist das Ergebnis. Der Wert deiner Arbeit liegt darin, zu wissen, *welche* Regeln tragen und unter welchen Bedingungen. Genau diese Information hat niemand, der nur Charts anschaut.

Ein Hinweis in eigener Sache: ICT-Konzepte sind nicht wissenschaftlich validiert, und es gibt für sie keine unabhängige Evidenzbasis. Das spricht nicht gegen sie — aber es bedeutet, dass die Beweislast bei dir liegt. Ein Ergebnis „funktioniert nicht“ ist deshalb genauso wertvoll wie ein positives: Es spart dir echtes Geld.

Wie es im Kopf bleibt

Das Kapitel, das dein eigentliches Ziel adressiert.



Der Grund, warum man ständig nachschlagen muss, ist fast nie mangelnde Intelligenz — es ist fehlende Wiederholung im richtigen Abstand. Fünf kurze Wiederholungen über einen Monat verteilt schlagen fünf Stunden am Stück deutlich.

Die vier Techniken, die du wirklich brauchst

1. Anki — aber nur eigene Karten

Nach jeder Lernsession legst du drei bis fünf Karten an. Keine fertigen Decks herunterladen: Das Formulieren der Karte ist der halbe Lerneffekt. Gute Karten sind Fragen, keine Definitionen — „Wie berechne ich in pandas einen 20-Perioden-Durchschnitt?“ statt „Was ist ein Moving Average?“. Zehn Minuten täglich reichen.

2. Der Blank-Page-Test

Jeden Sonntag: leeres Blatt, 10 Minuten, alles aufschreiben, was du diese Woche gelernt hast — ohne Unterlagen. Danach vergleichen. Die Lücken sind deine Anki-Karten für die nächste Woche.

3. Die 20-Minuten-Regel

Wenn du feststeckst: 20 Minuten ehrlich selbst versuchen, dann erst nachschlagen. Und dann — der entscheidende Teil — eine Anki-Karte daraus machen. So schlägst du dieselbe Sache nie ein drittes Mal nach.

4. Erst selbst bauen, dann die Bibliothek nutzen

Bevor du `sklearn.LinearRegression` verwendest, implementierst du eine lineare Regression einmal selbst in NumPy. Bevor du eine Backtest-Bibliothek nimmst, schreibst du eine primitive eigene. Danach ist die Bibliothek keine Blackbox mehr, sondern eine Abkürzung — und du merkst, wenn sie etwas anderes tut, als du denkst.

Was nicht funktioniert: Videos schauen und dabei nicken. Tutorials mitprogrammieren, ohne selbst zu variieren. Notizen markieren. Ein neues Framework ausprobieren, weil das aktuelle sich schwer anfühlt. Alle vier fühlen sich nach Fortschritt an und erzeugen keinen. Der ehrlichste Indikator für echtes Lernen ist Anstrengung beim Abrufen.

Kurse, Bücher, Werkzeuge

Alles Verlinkte ist geprüft. Fast alles ist kostenlos.

Python und Handwerk (Phase 0-1)

RESSOURCE	ART	WOFÜR UND LINK
CS50P — Harvard	kostenlos	Dein Hauptkurs. Zehn Wochen, exzellente Problem Sets. cs50.harvard.edu/python/
CS50P auf edX	kostenlos*	Gleicher Inhalt, mit Zertifikat gegen Gebühr. edx.org/learn/python/harvard-university-cs50-s-introduction-to-programming-with-python
Exercism — Python Track	kostenlos	Übungsaufgaben mit menschlichem Feedback. Ideal für die Freitags-Slots. exercism.org/tracks/python
Learn Git Branching	kostenlos	Git visuell in wenigen Stunden verstehen. learngitbranching.js.org
pandas — Getting Started	kostenlos	Offizielle Einführung, besser als die meisten Kurse. pandas.pydata.org/docs/getting_started/index.html
NumPy für Einsteiger	kostenlos	numpy.org/doc/stable/user/absolute_beginners.html
pytest-Dokumentation	kostenlos	Erste eigene Tests schreiben. docs.pytest.org

Mathematik (Phase 0 und 2)

RESSOURCE	ART	WOFÜR UND LINK
3Blue1Brown — Essence of Linear Algebra	kostenlos	Die beste visuelle Einführung, die es gibt. Zuerst schauen, immer. 3blue1brown.com/topics/linear-algebra
3Blue1Brown — Essence of Calculus	kostenlos	Analysis anschaulich, für den Gradienten-Teil. 3blue1brown.com/topics/calculus
Khan Academy — Linear Algebra	kostenlos	Zum Rechnen und Üben, mit Aufgaben. khanacademy.org/math/linear-algebra
Serlo Mathe (deutsch)	kostenlos	Für die Schulmathe-Auffrischung in Phase 0, auf Deutsch. de.serlo.org/mathe
Mathematics for Machine Learning	kostenlos	Deisenroth/Faisal/Ong, PDF legal frei. Kapitel 2-6 sind dein Zielumfang — als Nachschlagewerk, nicht als Lesebuch. mml-book.github.io
MIT 18.06 — Gilbert Strang	kostenlos	Der Klassiker. Optional, wenn du mehr Tiefe willst. ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/

Statistik und Zeitreihen (Phase 3)

RESSOURCE	ART	WOFÜR UND LINK
StatQuest (Josh Starmer)	kostenlos	Statistik und ML in verdaulichen Videos. Deine erste Anlaufstelle bei jedem unklaren Begriff. youtube.com/@statquest
Khan Academy — Statistics & Probability	kostenlos	khanacademy.org/math/statistics-probability
Seeing Theory	kostenlos	Wahrscheinlichkeit interaktiv und visuell. seeing-theory.brown.edu
Forecasting: Principles and Practice (Python)	kostenlos	Hyndman u. a., Python-Ausgabe. Dein Zeitreihen-Standardwerk, Kapitel 1–9. otexts.com/fpppy
statsmodels — tsa	kostenlos	ADF-Test, ACF/PACF, ARIMA in Python. statsmodels.org/stable/tsa.html

Machine Learning (Phase 4)

RESSOURCE	ART	WOFÜR UND LINK
An Introduction to Statistical Learning with Python (ISLP)	kostenlos	Das beste ML-Buch für deinen Zweck. PDF legal frei, mit Python-Labs. statlearning.com
INRIA scikit-learn MOOC	kostenlos	Vollständiger Praxiskurs mit Notebooks und Quizzes. Enthält ein eigenes Kapitel zu nicht-i.i.d.-Daten — genau dein Fall. inria.github.io/scikit-learn-mooc
Machine Learning Specialization (Andrew Ng)	Abo	Optional. Gut für Intuition, wenn ISLP zu trocken wird. coursera.org/specializations/machine-learning-introduction
scikit-learn User Guide	kostenlos	Nachschlagewerk, ungewöhnlich gut geschrieben. scikit-learn.org/stable/user_guide.html

Quant, Backtesting und Financial ML (Phase 5)

RESSOURCE	ART	WOFÜR UND LINK
Advances in Financial Machine Learning	Buch	López de Prado. Das Standardwerk für Labeling, Purged CV und Backtest-Overfitting. Kein Einsteigerbuch — ab Phase 5 lesbar, dann unverzichtbar.
Quantitative Trading (Ernest Chan)	Buch	Deutlich zugänglicher. Gut als Einstieg in Phase 5.
backtesting.py	kostenlos	Einfachste Backtest-Bibliothek. Ideal als zweiter Schritt, nachdem du eine eigene primitive Engine gebaut hast. kernc.github.io/backtesting.py
vectorbt	kostenlos*	Sehr schnell für Parameterstudien. Achtung: bildet Ausführung unrealistisch ab. vectorbt.dev

NautilusTrader	kostenlos	Event-getrieben, realistische Ausführung, gleicher Code für Backtest und live. Steile Lernkurve — erst in Phase 5 sinnvoll. nautilustrader.io
QuantConnect — Doku	kostenlos	Cloud-Plattform mit Daten. Die Doku ist frei, das Learning Center erfordert ein Konto. quantconnect.com/docs/v2/writing-algorithms
yfinance	kostenlos	Kursdaten zum Üben. Für Intraday-ICT-Arbeit später bessere Quellen nötig. github.com/ranaroussi/yfinance

* Grundfunktionen kostenlos, erweiterte Funktionen oder Zertifikate kostenpflichtig.

Werkzeuge, die du sofort einrichten solltest: Anki (Wiederholung, kostenlos, apps.ankiweb.net), VS Code als Editor, GitHub-Konto für dein Projekt-Repository, und ein einfaches Lerntagebuch — eine Textdatei genügt, in der du nach jeder Session zwei Zeilen notierst.

Meilensteine und Realitätscheck

Zum Abhaken — und zum ehrlich bleiben.

Deine Checkliste

BIS WANN	WORAN DU ERKENNST, DASS ES ERLEDIGT IST
☐ Woche 6	CS50P Woche 0–4 abgeschlossen, Anki läuft täglich, Schulmathe aufgefrischt
☐ Woche 12	CS50P komplett inklusive Abschlussprojekt, Git und pytest im Griff
☐ Woche 13	Meilenstein: Backtest v0 — Repository mit Datenladen, einfacher Regel, Equity-Kurve
☐ Woche 20	Lineare Algebra sitzt, Gradient Descent selbst implementiert
☐ Woche 25	Meilenstein: Feature-Bibliothek — ICT-Konzepte als getestete Funktionen
☐ Woche 30	Statistik und Zeitreihen durch, Kennzahlen selbst berechnet
☐ Woche 41	Meilenstein: ML-Modell v1 — schlägt oder schlägt nicht das Basismodell, sauber validiert
☐ Woche 48	Walk-Forward-Validierung und realistische Kosten im Backtest
☐ Woche 53	Meilenstein: Paper-Trading — Modell läuft live auf Demokonto mit Monitoring

Vier Warnsignale, bei denen du nachsteuern musst

Tutorial-Falle

Du hast drei Kurse angefangen und keinen beendet. Gegenmittel: Nur ein Kurs gleichzeitig, und erst der nächste, wenn der aktuelle abgeschlossen ist.

Werkzeug-Karussell

Du wechselst Bibliothek oder Framework, weil das aktuelle sich schwierig anfühlt. Schwierig ist normal. Wechsle nur bei einem konkreten technischen Grund.

Schönrechnen

Du optimierst Parameter, bis die Kurve gut aussieht. Gegenmittel: Testzeitraum festlegen, bevor du optimierst, und nicht anfassen.

Stiller Rückstand

Drei Wochen kein Sonntags-Review. Gegenmittel: Der Sonntag ist der letzte Termin, der ausfällt — notfalls auf 20 Minuten kürzen, aber nie streichen.

Was nach 12 Monaten realistisch ist — und was nicht

Das wirst du können

Eigenständig Python-Projekte bauen und testen. Finanzdaten verarbeiten. Mathematische Notation in Papers lesen. ML-Modelle korrekt trainieren und validieren. Einen Backtest bauen, dem man trauen kann. Deine eigenen Handelskonzepte in überprüfbare Regeln übersetzen und ehrlich bewerten.

Das wirst du nicht können

Bei einem Hedgefonds als Quant Researcher anfangen — dafür fehlen Stochastik, Optimierungstheorie und meist ein einschlägiger Abschluss. Zuverlässig Geld verdienen: Ein validiertes Modell ist eine Voraussetzung, keine Garantie. Und du wirst weiterhin nachschlagen — nur eben gezielt statt ratlos.

Der Unterschied zwischen jemandem, der sich Quant nennen darf, und jemandem, der es nicht darf, ist nicht die Menge an Wissen. Es ist die Fähigkeit, dem eigenen Ergebnis zu misstrauen und trotzdem zu einer Entscheidung zu kommen.

Dieses Dokument ist ein Lernplan, keine Anlageberatung. Handelsentscheidungen und das damit verbundene Risiko liegen allein bei dir. Alle Links wurden am 24. August 2026 geprüft.